

Norma Maestra de Ingeniería de Software e Integración

Programas, interconexiones, CORE, documentación, UAT y control de cambios

Documento	SITC-NOM-001
Versión	v4.1
Estado	ESTÁNDAR OBLIGATORIO / NORMA ACTIVA
Fecha	22 de septiembre de 2026
Sustituye / absorbe	SITC-NOM-001 v4.0. Incorpora la separación entre ECOSYSTEM_INTERCONNECTION (gobernada por CORE/SITC) y EXTERNAL_CONNECTION (gobernada por el Sistema propietario en cada Instancia PE).
Audiencia	Equipo de Sistemas Cajamarca, desarrolladores, QA y ChatGPT utilizados por el equipo

REGLA CENTRAL: ningún programa se desarrolla o modifica “en vacío”. Todo trabajo parte de esta norma, del SCENARIO_SNAPSHOT vigente y, si existe, del ZIP/código actual del programa.

Principio de configuración. El código implementa el programa; los *.md explican el programa; el *.sitcpack describe su arquitectura portable. CORE gobierna Sistemas e interconexiones del ecosistema; la configuración operacional de conexiones externas vive en el Sistema propietario por Instancia PE.

1. Alcance, audiencia y fuentes de verdad

Esta norma establece una forma única de diseñar, construir, probar, versionar, documentar, integrar, revisar y transferir software dentro del Ecosistema Cajamarca. Debe permitir que distintos desarrolladores y chats produzcan entregables compatibles, trazables y transferibles sin depender de memoria tácita.

1.1 Fuentes de verdad y precedencia

Orden	Fuente	Aplicación
1	CORE VIGENTE + SCENARIO_SNAPSHOT vigente	Catálogo/arquitectura aprobada de Sistemas, Interconexiones y versiones.
2	Baseline congelada del programa	ZIP/código + docs/DECISIONS/MASTER/ARCHITECTURE y contratos vigentes.

3	Decisión/CR formal aprobado para la intervención	Autoriza la evolución concreta sin reescribir la historia.
4	SITC-NOM-001 v4.1	Estándar técnico, documental y de control de cambios.
5	Preferencias locales	Solo aplican si no contradicen los niveles anteriores.

- Una baseline aprobada es inmutable. Un cambio crea una nueva versión/RC/vertical reabierta.
- Una instrucción informal, mensaje de chat o conveniencia del programador no sustituye una decisión/CR cuando la categoría es controlada.
- Si dos artefactos activos se contradicen y no existe una regla de supersesión clara, detener la normalización de ese punto y elevar la contradicción.

1.2 Vocabulario obligatorio

Término	Definición
Programa	Unidad lógica de software identificada por un Program ID estable.
Program ID	Identificador canónico usado en CORE, *.sitcpack, documentación y contratos.
NATIVO	Sistema construido/modificable bajo este estándar.
LEGADO	Sistema existente que no fue construido bajo el estándar o cuyo código no está disponible/modificable directamente.
Interconexión	Relación técnica con dos clases: ECOSYSTEM_INTERCONNECTION entre Program ID del ecosistema, gobernada por CORE/SITC; y EXTERNAL_CONNECTION entre un Programa y un tercero, gobernada por el Sistema propietario en su Instancia PE.
Interface	Endpoint REST, evento, topic, webhook, archivo u otro punto técnico concreto de una interconexión.
System of Record	Programa autoritativo de un dato o entidad.
Vertical	Corte funcional acotado, comprobable, versionable y congelable.
SCENARIO_SNAPSHOT	*.sitcpack que representa la arquitectura/catálogo de partida.
COMPONENT_DELTA	*.sitcpack con cambios introducidos por un programa/vertical.
Instancia PE	Contexto país-empresa/operativo donde CORE materializa valores efectivos por ambiente.

2. Principios no negociables

- No programar una ECOSYSTEM_INTERCONNECTION sin declararla primero en lenguaje humano y en el *.sitcpack de trabajo.
- No crear una EXTERNAL_CONNECTION sin documentar tercero, propósito, contrato, auth, resiliencia, auditoría y configuración en el Sistema propietario.
- No consumir directamente la base de datos de otro programa como patrón normal; las excepciones legacy deben ser explícitas, aprobadas y documentadas.
- No hardcodear host, IP, puerto, credencial, URL física o endpoint físico dentro de la lógica de negocio.
- Cada dato compartido debe tener un System of Record explícito; todo Sistema NATIVO usa componentes reutilizables de integración, no clientes ad hoc.

- No incluir secretos efectivos en código portable, documentación portable o *.sitcpack.
- Una vertical congelada no se modifica silenciosamente.
- Cada entrega mantiene código, documentación, migraciones y arquitectura sincronizados.
- Promover arquitectura a VIGENTE no modifica automáticamente valores productivos de Instancias PE.

3. Catálogo de Sistemas y metadatos maestros

Campo	Regla
programId	Obligatorio; UPPER_SNAKE_CASE o sigla estable. No renombrar por razones cosméticas.
name/displayName	Nombre visible; no sustituye programId.
type/scope	UNIVERSAL o PE_SPECIFIC según el modelo vigente de CORE.
origin	NATIVO o LEGADO. Sustituye vocabularios históricos como NEW_AI/LEGACY.
version	Versión maestra del programa cuando aplique.
status/lifecycle	Estado vigente conforme al catálogo CORE; no inferirlo por nombre de carpeta.
technology	Lenguajes, frameworks, versiones y componentes relevantes.
initialConfiguration	Plantilla de parámetros necesarios para inicializar el Sistema por Instancia PE + ambiente. No incluye conexiones externas concretas, que son configuración operacional del Sistema propietario.

El catálogo canónico dinámico vive en CORE y viaja a los chats mediante SCENARIO_SNAPSHOT; no mantener una tabla estática de Program IDs dentro de esta norma.

4. Estándar técnico canónico

Área	Estándar
Frontend web	TypeScript 6 + React 19.2 + Vite
Backend	Java 25 LTS + Quarkus
APIs	REST + OpenAPI + versionamiento /api/v1/...
Persistencia	Hibernate
Migraciones	Flyway; reproducibles y versionadas
Base de datos	PostgreSQL 17
UAT/desarrollo	Docker + Docker Compose
Arquitectura inicial	Modular monolith; evitar microservicios prematuros
Identificadores	UUID internos + códigos humanos estables cuando corresponda
Auditoría	Obligatoria para operaciones relevantes
Borrado	Preferir estados/inactivación/soft-delete; evitar DELETE físico

Identidad/autorización	IDENT
Aislamiento	instance_country_id y contexto de Instancia PE
CAJ-MOBILE	React Native cuando convenga compartir Android/iOS
CAJ-ANDROID-LITE	Kotlin nativo + Android SDK + Views/XML cuando domine rendimiento/compatibilidad

No sustituir este stack por otro por preferencia local. Las excepciones deben aprobarse, documentarse en ARCHITECTURE/DECISIONS y reflejarse en el *.sitcpack.

5. Modelo de desarrollo por verticales

Paso	Regla
1. Definir	Código/nombre, propósito, alcance, fuera de alcance y criterios de aceptación.
2. Revisar baseline	Leer norma, SCENARIO_SNAPSHOT y ZIP/documentación vigente.
3. Diseñar	Modelo, UI, reglas, APIs, SoR, impactos, seguridad y migraciones.
4. Declarar integraciones	Distinguir ECOSYSTEM_INTERCONNECTION (declarar antes, asignar IDs y actualizar delta) de EXTERNAL_CONNECTION (documentar contrato y configuración local del Sistema).
5. Construir	Implementar solo la vertical activa y reutilizar componentes estándar.
6. Empaquetar	Nueva versión UAT/RC con repo/ y documentación sincronizada.
7. UAT	Probar criterios funcionales, integraciones, errores y compatibilidad.
8. Corregir	Cada corrección produce una nueva entrega; nunca reemplazar silenciosamente.
9. Congelar	Con aprobación, actualizar UAT/DECISIONS/CHANGELOG y fijar baseline.
10. Siguiente vertical	Partir de la última baseline congelada.

5.1 Estados de vertical

Estado	Significado
EN_DESARROLLO	Diseño o implementación activa.
UAT	Entrega disponible para prueba.
CONGELADA	Aprobada; baseline funcional protegida.
REABIERTA	Vertical congelada cuya evolución fue autorizada formalmente.

6. Baseline, Release Candidate y control de cambios

- BASELINE: artefacto aprobado e inmutable de referencia.

- RC: nueva derivación identificable de la baseline. Convención de flujo: BASELINE -> RC1 -> RC2 -> ... -> PROD.
- Una intervención debe concentrarse en una sola vertical; hallazgos de otra vertical se registran y se abren aparte.
- Todo cambio real se registra en CHANGELOG-SISTEMAS.md o docs/CHANGELOG.md canónico.

6.1 Clasificación de cambios

Clase	Categorías	Regla
PERMITIDO	Bugs; responsive/UI polish; accesibilidad; performance; logging/monitorización; CI/CD; seguridad técnica que no rediseña permisos/roles/workflows; micro-interacciones UX.	Sistemas puede implementar dentro de la vertical; debe registrar y probar.
REQUIERE APROBACIÓN	Infraestructura productiva; reglas de negocio; workflow; modelo de datos; APIs/contratos; integraciones; nuevos estados/catálogos; arquitectura; nuevas dependencias materiales.	Debe existir decisión/CR antes de implementar. Incluir compatibilidad, migración y rollback cuando aplique.
PROHIBIDO COMO CAMBIO LOCAL	Renombrar Program IDs/nomenclatura protegida; cambiar SoR; alterar decisiones arquitectónicas congeladas por conveniencia; sobrescribir baseline.	No puede resolverse mediante “ajuste local”. Requiere una decisión arquitectónica de nivel maestro/supersesión y nueva versión de los artefactos afectados.

Seguridad técnica incluye endurecimiento, dependencias, secretos/configuración y controles equivalentes; no autoriza rediseñar roles, permisos funcionales o flujos de negocio.

7. Registro documental obligatorio

Cada ZIP debe ser autosuficiente para que otro desarrollador o ChatGPT continúe el trabajo sin reconstruir decisiones desde la conversación.

Archivo	Contenido mínimo
repo/README.md	Arranque rápido, propósito, prerequisites, URL y estructura.
docs/00_HANDOFF.md	Ruta de lectura y handoff ejecutivo/técnico.
docs/MASTER.md	Contexto funcional, reglas, verticales y pendientes.
docs/ARCHITECTURE.md	Arquitectura técnica, límites, SoR y patrón de integración.
docs/DECISIONS.md	Decisiones vigentes, superseded/deprecated y fecha.
docs/CHANGELOG.md	Historial de entregas, cambios, archivos/migraciones y validación.
docs/UAT.md	Criterios, casos, resultados y freeze.
docs/SITCPACK.md	Snapshot de partida, deltas, interconexiones y archivos entregados.
docs/API_CATALOG.md	APIs/eventos, request/response, auth, contractVersion e IDs.
docs/INTERCONNECTIONS.md	Interconexiones completas, estados, pendientes legacy y pruebas.
docs/SECURITY.md	AuthN/AuthZ, secretos, exposición de red y controles.

docs/KNOWN_LIMITATIONS.md	Limitaciones, deuda y funciones diferidas.
docs/ECOSYSTEM_IMPACT.md	Obligatorio cuando haya impacto multi-sistema; resumen cuantificado y estado end-to-end.
docs/NEXT_ACTIONS.md	Secuencia exacta de chats/acciones y archivos a transportar.
CHANGE_REQUEST_<TARGET>.md	Uno por cada Program ID propio que requiera modificación.

No dejar decisiones críticas solo en el chat, commits sin descripción o memoria del programador.

7.1 Changelog mínimo

<PROGRAM_ID> - <VERTICAL> - <VERSION/RC> - <AAAA-MM-DD>

Baseline: <nombre exacto>

Responsable: <persona/equipo>

Cambios realizados

- <ID> | PERMITIDO/APROBADO | <descripción>

Archivos / migraciones

Integraciones / SITC

Validación: build / tests / UAT / pendientes / riesgos

8. Modelo rector: CORE, SITC y *.sitcpack

Elemento	Responsabilidad
CORE	Registro/resolver de Sistemas y ECOSYSTEM_INTERCONNECTION; mantiene definiciones, versiones, plantillas y bindings por Instancia PE + ambiente. No administra conexiones externas concretas.
*.sitcpack	Formato portable de intercambio. Representa SCENARIO_SNAPSHOT o COMPONENT_DELTA.
SITC	Modelo/estándar de arquitectura, escenarios y trazabilidad. No crea nomenclatura paralela.
Sistema iniciador	Para ecosistema resuelve vía CORE/cache; para terceros usa la configuración operacional del Sistema propietario y ejecuta la conversación real.
Sistema destino	Expone/aplica el contrato de negocio y sus reglas de dominio.

CORE gobierna las interconexiones entre Program ID del ecosistema. Las conexiones con terceros, dispositivos o servicios externos son gobernadas por el Sistema propietario dentro de cada Instancia PE.

8.1 Definición maestra vs. valor efectivo

Capa	Contiene	Dónde vive
------	----------	------------

Arquitectura Sistemas, ECOSYSTEM_INTERCONNECTION, versiones, tecnologías e initialConfiguration (plantilla). COMPONENT_DELTA -> CORE VIGENTE o ESCENARIO		SCENARIO_SNAPSHOT /
Operación PE Bindings efectivos de ECOSYSTEM_INTERCONNECTION por Instancia PE + ambiente; las EXTERNAL_CONNECTION y sus endpoints/credenciales se administran en el Sistema propietario. CORE para ecosistema; Sistema propietario para externos		
Secretos	Referencias a secreto; nunca el secreto portable.	Vault/secret store + credential_ref

9. Nomenclatura canónica de ECOSYSTEM_INTERCONNECTION

Objeto	Formato canónico	Ejemplo
Program ID	UPPER_SNAKE_CASE / sigla estable	ATS, SGI_COM, SIC_RRHH
Referencia estable	<ORIGEN>_<DESTINO>_<NNNN>	ATS_SGI_COM_0001
ID versionado	<REFERENCIA>_v<NNN>	ATS_SGI_COM_0001_v001
Versión maestra	v<NNN>	v001
Interface ID	<REFERENCIA>_IF<NN>	ATS_SGI_COM_0001_IF01
Versión de contrato	v1, v2, ...	v1

- La referencia estable no cambia durante la vida lógica de la interconexión.
- La secuencia usa cuatro dígitos y se asigna por par direccional ORIGEN -> DESTINO.
- El código de negocio pinnea el ID versionado para comportamiento determinista.
- v001/v002 identifica revisión maestra; contractVersion v1/v2 identifica compatibilidad del contrato y evoluciona independientemente.
- IDs históricos con INT_, doble guion bajo, cinco dígitos o V0001 se conservan solo como alias/legacyConnectionId.

10. Módulo genérico de integraciones

Todo Sistema NATIVO debe usar un módulo reusable para integraciones salientes. Para ECOSYSTEM_INTERCONNECTION usa CoreResolver; para EXTERNAL_CONNECTION resuelve/configura localmente en el Sistema propietario. Transporte, auth, resiliencia, validación, telemetría y business adapters se reutilizan en ambos casos.

Componente lógico	Responsabilidad obligatoria
IntegrationExecutor	API única usada por lógica de negocio. Recibe ID versionado, payload y contexto.
CoreResolver	Resuelve en CORE por Instancia PE + ambiente únicamente una ECOSYSTEM_INTERCONNECTION.
ResolutionCache	Cachea resolución válida de ecosistema y, cuando aplique, estado/configuración local de conectores externos; TTL/invalidation documentados.
Transport	REST/HTTP, evento, archivo u otro transporte aprobado. No se duplica por integración.
Authentication	Aplica service identity/credential_ref sin exponer secretos a negocio.

Resilience	Timeout, retry, backoff, circuit handling e idempotencia cuando aplique.
ContractValidator	Valida request/response/schema y contractVersion cuando aplique.
Audit/Telemetry	Correlation ID, latencia, resultado, logs y métricas.
Business Adapter	Transformación específica de payload. Nunca reimplementa resolver/auth/cache/transporte.

```
executeIntegration(
  "ATS_SGI_COM_0001_v001",
  requestData,
  context
)
```

- Para ECOSYSTEM_INTERCONNECTION, una indisponibilidad puntual de CORE no debe derribar la operación si existe una resolución cacheada válida.
 - Para EXTERNAL_CONNECTION, la resolución/configuración es local al Sistema propietario; agregar o cambiar bancos, cámaras u otros terceros no requiere CORE salvo cambio de capacidad arquitectónica.

11. Contrato exhaustivo de ECOSYSTEM_INTERCONNECTION

EXTERNAL_CONNECTION se documenta con contrato equivalente dentro del Sistema; no se registra como Program ID ni binding CORE.

Campo	Debe quedar definido
Identidad	Referencia estable, ID versionado, nombre, revisión, estado, fecha y dueño técnico.
Participantes	Source, target, initiator y módulos origen/destino.
Propósito/trigger	Qué necesidad resuelve y qué evento/acción inicia la conversación.
System of Record	Dueño de cada dato relevante.
Patrón	REST síncrono, evento asíncrono, archivo u otro patrón aprobado.
Contrato	Método/path lógico o evento/topic, OpenAPI/schema, request, response y códigos de error.
Seguridad	IDENT/service identity, scopes/roles, TLS, credential_ref.
Resiliencia	Timeout, retries, backoff, circuit handling e idempotencia.
Cache	Qué resolución se cachea, TTL e invalidación.
Auditoría/observabilidad	Correlation ID, actor/sistema, timestamps, resultado, logs y métricas.
Datos	Clasificación, minimización y snapshots históricos cuando corresponda.
Compatibilidad	contractVersion, breaking changes, deprecación y transición.
Dependencias	Cambios requeridos en source, target y/o intervención legacy.
Estado end-to-end	READY / UAT / MANUAL_PENDING / BLOCKED / ACTIVE según realidad.

12. Sistemas NATIVOS y LEGADOS

Escenario	Regla
Destino LEGADO modificable	Aplicar módulo estándar donde sea viable; documentar adaptaciones y compatibilidad.
Destino LEGADO no modificable	Implementar resolver/adaptador/llamada del lado Cajamarca; no inventar cambios en el legacy. Documentar contrato esperado y estado MANUAL_PENDING/BLOCKED.
Ambos NATIVOS	Si el proveedor requiere cambio, trabajar/entregar ambos lados: consumidor usa el ID y adapter; proveedor expone/ajusta contrato y aplica sus reglas de dominio.
Gateway/adapter	Permitido para encapsular una excepción legacy; no debe crear una segunda fuente de verdad ni ocultar el SoR real.

13. Contrato de *.sitcpack para CORE

Un *.sitcpack arquitectónico es un ZIP con extensión propia. Todo paquete que deba round-trippear sin pérdida por CORE usa cuatro archivos de raíz. Las conexiones externas concretas no se registran como interconexiones CORE; solo la capacidad/esquema del conector se refleja si cambia la arquitectura.

```
PACK.sitcpack
├─ manifest.json
├─ architecture.json
├─ core-normalized.json
└─ README.md
```

Archivo	RoI
manifest.json	Schema, packageType, packageId, fecha, programa/versión primaria, baselineFingerprint y descripción.
architecture.json	Representación portable SITC para interoperabilidad, trazabilidad y lectura humana/máquina.
core-normalized.json	Representación exacta CORE; preserva versiones, campos maestros e initialConfiguration sin reinterpretación.
README.md	Propósito, alcance, origen/baseline, contenido y notas de importación.

13.1 packageType y reglas de delta

Tipo	Uso	Regla
SCENARIO_SNAPSHOT	Arquitectura/catálogo de partida o vigente.	No confundir con valores operativos de una Instancia PE.
COMPONENT_DELTA	Cambios introducidos por un programa/vertical.	Debe incluir definiciones completas activas de los objetos afectados; no fragmentos ambiguos.

13.2 Configuración Inicial

- systems[].initialConfiguration define la plantilla de parámetros del Sistema para inicializar por Instancia PE + ambiente.
- interconnections[].initialConfiguration aplica solo a ECOSYSTEM_INTERCONNECTION; baseline común: PROTOCOL, HOST, PORT, BASE_PATH, AUTH_TYPE, CREDENTIAL_REF, TIMEOUT_MS, RETRIES.
- Las EXTERNAL_CONNECTION concretas se configuran operacionalmente en el Sistema propietario de cada Instancia PE; no crean bindings CORE.
- Un parámetro sensitive nunca incluye el secreto efectivo; usar credential_ref o un secret store seguro.
- Si cambia la plantilla/capacidad arquitectónica, el COMPONENT_DELTA incluye la definición completa activa del objeto afectado.
- Valores efectivos: bindings de ecosistema en CORE; conexiones externas en el Sistema propietario. Nunca inventarlos en el delta arquitectónico.

14. Importación, escenarios y promoción en CORE

1. Cargar SCENARIO_SNAPSHOT/COMPONENT_DELTA en CORE como ESCENARIO cuando se esté validando una evolución.
2. CORE valida schema, Program IDs, referencias estables, versiones y conflictos antes de aplicar.
3. La importación debe mostrar preview de crear / actualizar / sin cambio / conflicto / inválido.
4. Programs se resuelven por programId; no duplicar por nombre visible.
5. Interconexiones se resuelven por referenceCode estable + ID versionado.
6. Ante conflicto con una baseline activa, no sobrescribir silenciosamente; resolver en escenario.
7. Solo ADMIN promueve un escenario aprobado a VIGENTE.
8. La promoción arquitectónica no activa ni modifica automáticamente valores de Sistemas/Interconexiones en Instancias PE.
9. Después de promover, descargar el nuevo SCENARIO_SNAPSHOT VIGENTE y usarlo como baseline del siguiente trabajo.

15. Gate obligatorio cuando cambia una ECOSYSTEM_INTERCONNECTION

Control	Evidencia requerida
Identidad	Referencia estable + ID versionado anterior/nuevo.
Contrato	contractVersion anterior/nueva y justificación de breaking change si aplica.
Impacto	Program IDs afectados, SoR, initiator, source, target y cambios requeridos en contraparte.
Módulo genérico	Confirmar IntegrationExecutor/CoreResolver/cache para ecosistema; no cliente ad hoc ni endpoint hardcoded.
SITC	COMPONENT_DELTA válido con manifest.json, architecture.json, core-normalized.json y README.md.
Configuración Inicial	Si cambia la plantilla, incluir initialConfiguration completa del Sistema/Interconexión afectada.
Instancia PE	No incluir secretos ni valores productivos en delta. Bindings de ecosistema se operan por ambiente en CORE; configuración externa concreta permanece en el Sistema propietario y queda fuera de este gate.
Validación	Cargar en CORE como ESCENARIO, revisar preview/conflictos y no declarar VIGENTE sin aprobación ADMIN.

16. Empaquetado UAT y comandos

```
<ENTREGA>.zip
├─ repo/
│   ├── frontend/
│   ├── backend/
│   ├── database/      # si aplica
│   ├── docs/
│   ├── sitc/
│   ├── scripts/
│   │   ├── uat-start.ps1
│   │   └── uat-open.ps1
│   ├── docker-compose.yml
│   ├── .env.example
│   └── README.md
```

- repo/ es la única raíz interna del ZIP; no envolverla dentro de otra carpeta de versión.
- Incluir un *.sitcpack CURRENT sincronizado y uno versionado cuando se cierre una vertical o exista cambio arquitectónico material.
- Si no hubo cambio arquitectónico, declarar expresamente “Sin cambios de interconexiones/SITC” en el changelog.
- uat-start.ps1 debe detener correctamente la versión UAT anterior y levantar backend, frontend, DB y servicios requeridos.
- uat-open.ps1 abre la URL/punto de entrada esperado.
- Los comandos entregados al usuario deben incluir la ruta exacta; no inventar una ruta definitiva.

```
cd "<RUTA_EXACTA>"
powershell -ExecutionPolicy Bypass -File .\repo\scripts\uat-start.ps1
powershell -ExecutionPolicy Bypass -File .\repo\scripts\uat-open.ps1
```

17. Validación y gate pre-producción

- Ejecutar build, tests disponibles, arranque UAT, migraciones y regresión básica.
- Comparar BASELINE vs RC antes de UAT final/producción, incluyendo cambios no declarados en changelog.
- No declarar “apto para producción” si existen fallos críticos, cambios no autorizados o divergencias de arquitectura/SoR.

Resultado	Significado	Acción
APROBADO	Cambios permitidos/aprobados; arquitectura y reglas preservadas.	Puede pasar a UAT final y producción conforme al proceso.
APROBADO CON OBSERVACIONES	Sin violaciones críticas; quedan ajustes/deuda documentada.	Corregir antes de producción o aceptar formalmente la observación.
NO APROBADO	Cambios no autorizados, fallos críticos o desviaciones de arquitectura/SoR.	Corregir y entregar nuevo RC.

18. Change Request mínimo

CR-ID: CR-<PROGRAMA>-<####>

Programa / vertical:
 Solicitante:
 Problema / necesidad:
 Cambio propuesto:
 Impacto: UI | workflow | regla de negocio | BD | API | integración | arquitectura
 Sistemas afectados:
 Compatibilidad / migración:
 Riesgo / rollback:
 Aprobado por:
 Fecha de aprobación:

19. Orquestación multi-chat y handoff

La continuidad no termina cuando un programa compila o supera UAT. Al cerrar cada trabajo se identifica el impacto arquitectónico, se generan artefactos de handoff y se indica el orden exacto de los siguientes chats/acciones.

Momento	Obligación	Salida
Inicio	Partir de arquitectura VIGENTE y ZIP actual.	SCENARIO_SNAPSHOT VIGENTE + baseline programa.
Durante	Declarar/versionar cambio arquitectónico antes de implementarlo.	COMPONENT_DELTA de trabajo.
Cierre programa A	Cuantificar nuevas/modificadas/retiradas interconexiones y programas afectados.	ECOSYSTEM_IMPACT + COMPONENT_DELTA.
Programas B/C	Entregar instrucciones y CHANGE_REQUEST autosuficiente por destino.	CHANGE_REQUEST_<TARGET>.md + delta pertinente.
Consolidación	Cargar deltas en CORE como ESCENARIO y revisar conflictos/coherencia end-to-end.	Escenario validado.
Publicación	Solo ADMIN promueve el escenario.	Nuevo SCENARIO_SNAPSHOT VIGENTE.

19.1 Cierre obligatorio

- Impacto sobre el Ecosistema: cantidad + IDs de interconexiones nuevas/modificadas/retiradas; Program IDs que requieren cambios; estado end-to-end; artefactos.
- Sigüientes Acciones: “Ahora ve al chat de <PROGRAM_ID>”, archivos a adjuntar, cambios concretos, pruebas y criterios; repetir en orden y finalizar en CORE.
- Si no hay impactos externos, declararlo expresamente: otros programas = ninguno; interconexiones nuevas/modificadas/retiradas = 0.

20. Definition of Done - candidato a producción

- ☐ Cumple criterios funcionales/UAT de la vertical.
- ☐ No altera decisiones congeladas sin aprobación/supersesión formal.
- ☐ Build y pruebas disponibles pasan o sus fallos están explícitamente justificados.
- ☐ No existen errores críticos conocidos sin tratamiento.
- ☐ Migraciones son reproducibles y versionadas.

- ☐ Logs/manejo de errores aplicables están implementados.
- ☐ Permisos y controles existentes no fueron degradados.
- ☐ Changelog y documentación afectada están completos.
- ☐ SITC/*.*sitcpack está actualizado y válido si corresponde.
- ☐ RC puede desplegarse de forma repetible.
- ☐ Rollback/reversión está definido cuando el riesgo lo exige.
- ☐ BASELINE vs RC fue revisado antes de producción.
- ☐ Impacto multi-sistema y siguientes acciones están documentados.

21. Checklist rápido para Sistemas y ChatGPT

ANTES DE EDITAR	AL ENTREGAR
Adjuntar norma + SCENARIO_SNAPSHOT + ZIP baseline.	Entregar ZIP RC/UAT nuevo; nunca sobrescribir baseline.
Indicar Program ID, vertical y cambio solicitado.	Actualizar changelog/documentación/migraciones.
Leer README/MASTER/DECISIONS/ARCHITECTURE/SITC.	Entregar *.*sitcpack válido si hubo cambio arquitectónico.
Clasificar el cambio: permitido / aprobación / prohibido local.	Reportar build/tests/UAT, riesgos y archivos modificados.
Declarar interconexiones antes de programarlas.	Comparar BASELINE vs RC y emitir estado pre-producción.
No mezclar otra vertical.	Cerrar con Impacto sobre Ecosistema + Siguiendo Acciones.

22. Supersesión y transición

- SITC-NOM-001 v4.1 sustituye SITC-NOM-001 v3.2 como norma técnica vigente.
- CSES-DEV-001 v1.1 queda ABSORBIDO y RETIRADO como documento independiente; sus controles de baseline/RC, clasificación, changelog, CR, gate pre-producción y Definition of Done viven ahora en esta norma.
- CAJ-STD-IA-DEV-001 v2.0 queda como protocolo personal de trabajo con ChatGPT y no debe duplicar esta norma exhaustiva.
- Los anexos históricos y ejemplos con formatos de ID obsoletos se conservan solo en archivo histórico, no como norma activa.
- Ante un conflicto con documentación histórica, prevalece la regla activa de esta v4.1 y la arquitectura VIGENTE aprobada en CORE.

Objetivo final: cualquier miembro del equipo debe poder recibir el ZIP de un programa + su *.*sitcpack + esta norma y entender qué existe, cómo se cambia, cómo se integra, cómo se prueba y qué queda pendiente sin depender de la conversación original.